

Multimodal Fashion RAG — гайд для НОВИЧКОВ

Этот документ объясняет проект с нуля: сначала — что вообще значат все слова (гlossарий), потом — как это собрано в систему, и наконец — какие решения приняты и почему.

Читать сверху вниз. Если термин непонятен — он почти наверняка объяснён в Части 1.

Часть 1. Глоссарий — все термины простыми словами

1.1 RAG (Retrieval-Augmented Generation)

Проблема: LLM (типа ChatGPT) знают только то, что видели в обучении. Про твой каталог товаров она ничего не знает.

Решение: RAG = "найди релевантные документы → дай их LLM как контекст → LLM ответит, опираясь на них".

Три буквы:

- **R (Retrieval)** — поиск. Берём запрос пользователя, находим top-N релевантных документов.
- **A (Augmented)** — дополненная. Найденные документы подставляем в промпт.
- **G (Generation)** — генерация. LLM пишет человеческий ответ на основе контекста.

В нашем проекте ядро — это R (retrieval). G — опциональная обёртка сверху.

1.2 Эмбеддинг (embedding / vector)

Эмбеддинг — это текст/картинка, превращённые в **список чисел** (вектор). Например: 512 чисел.

Идея: если два объекта похожи по смыслу — их векторы близки в пространстве (малый угол между ними). Если разные — далеки.

Пример: вектор слова "кот" $\approx$ вектор слова "кошка", но далеко от "трактор".

Измерение близости — обычно **косинусное сходство** (косинус угла между векторами, от -1 до 1, ближе к 1 — ближе по смыслу).

1.3 Dense vs Sparse векторы

Dense (плотный) вектор:

- Длина фиксированная: например, 512 чисел.
- **Каждое** число ненулевое, это float.
- Получается из нейросети (BERT, CLIP и т.д.).
- **Ловит смысл:** "кофта" и "свитер" будут близки, даже если в документах нет общих

слов.

Sparse (разреженный) вектор:

- Длина = размеру словаря (десятки тысяч).
- **Почти все числа — нули**, ненулевые только у слов, которые есть в тексте.
- Классика: **BM25** (формула релевантности из поисковиков 90-х).
- Новое: **BM42** — то же самое, но вес слов берётся из attention-весов трансформера (Qdrant так делает).
- **Ловит точные слова**: артикулы, бренды, редкие термины.

Важно: ни один вектор сам по себе не идеален. Dense упускает точные совпадения ("iPhone 15 Pro Max" — конкретика). Sparse упускает синонимы ("кофта" vs "свитер"). Поэтому их комбинируют — это и есть **гибридный поиск**.

1.4 Vector DB (векторная база) — Qdrant

Обычная БД (PostgreSQL) умеет искать по точному совпадению ("WHERE color = 'red'").

Векторная БД умеет искать по близости векторов: "найди 10 точек, чьи векторы ближе всего к моему запросу".

Qdrant — одна из таких БД (написана на Rust, open-source, self-hosted).

Что умеет Qdrant (и чем хорош для нас):

- Хранит точки с **несколькими именованными векторами** (dense + sparse) в одной записи — очень удобно.
- Умеет **hybrid search** на сервере: отправляешь оба вектора, он сам объединяет.
- Поддерживает **фильтры по payload** (метаданные): "ищи близко по вектору, но только где color='red'".

1.5 RRF (Reciprocal Rank Fusion)

Проблема: у нас два поиска (dense и sparse), у каждого свой top-20. Как объединить в один рейтинг?

Наивный подход: сложить скоры. **Не работает**, потому что у dense скор 0-1, а у BM25 может быть 0-50. Шкалы разные.

RRF — умный приём: **забыть про скоры, смотреть только на РАНГ** (позицию в списке).

Формула: для каждого документа $score = 1 / (k + rank_dense) + 1 / (k + rank_sparse)$ (k обычно = 60).

Почему работает:

- Если документ в топе обоих списков — суммарный скор высокий.
- Скорость, скорость, скорость: не надо тюнить веса.
- Устойчиво к разнице масштабов.

Qdrant делает это на сервере.

1.6 Bi-encoder vs Cross-encoder

Bi-encoder (два энкодера):

- Берём запрос → превращаем в вектор (отдельно).
- Берём документ → превращаем в вектор (отдельно).
- Сравниваем косинусом.
- **Быстрый** (векторы документов предвычислены, при поиске — только запрос).
- **Менее точный**: сравнивает два сжатия, детали теряются.

Cross-encoder (один энкодер):

- Подаём в модель **пару** (запрос + документ) целиком, разделённые спецсимволом.
- Модель выдаёт один скор — "насколько релевантно".
- **Точнее**: модель видит взаимодействие слов запроса и документа.
- **Медленнее**: документы нельзя предвычислить, считаем заново каждый раз.

Отсюда — двухступенчатый поиск:

1. Bi-encoder быстро достаёт **top-20 кандидатов** из миллиона документов.
2. Cross-encoder пересортирует **только эти 20**, выдаёт точный топ-5.

Это и называется **reranking**.

1.7 CLIP (Contrastive Language-Image Pretraining)

Модель от OpenAI (2021), которая **учит текст и картинки в одном векторном пространстве**.

Идея обучения: берём миллиарды пар (картинка, подпись из интернета), тренируем две сети (одна для картинок, одна для текста), чтобы:

- вектор картинки $\approx$ вектор её подписи,
- вектор картинки $\neq$ вектор чужой подписи.

Что получаем:

- Запрос "black dress" → вектор в том же пространстве, что и вектор картинки платья → их можно сравнивать.
- **Zero-shot**: модель не дообучалась на fashion, но всё равно находит платья по запросу — потому что видела их в общих данных.

Multilingual CLIP — дообученная версия, где text-encoder понимает много языков (включая русский).

1.8 Мультимодальность

"**Модальность**" — тип данных (текст, картинка, аудио, видео).

Мультимодальная модель обрабатывает несколько типов одновременно. CLIP мультимодальный (текст + картинки).

Зачем нам: один индекс обслуживает два разных входа — поиск по описанию и поиск по картинке. Без мультимодальности пришлось бы держать два отдельных индекса.

1.9 LLM и что такое токены/контекст

LLM (Large Language Model) — большая языковая модель, типа GPT-4, Llama, Claude. Предсказывает следующее слово.

Токен — кусочек слова (обычно 1 токен $\approx$ 0.75 слова английского, или 3-4 символа). LLM работает в токенах.

Контекст — максимум токенов, которые модель видит за один запрос (system prompt + user message + ответ). У llama3.1-8b — 8к токенов.

API-шный LLM (Cerebras, OpenAI, Groq) — ты делаешь HTTP-запрос, получаешь ответ. Плюс — не надо держать модель у себя. Минус — зависимость от сервиса.

Cerebras в нашем проекте: хостит llama3.1-8b, **2000+ токенов/сек** — настолько быстро, что LLM перестаёт быть узким местом.

1.10 Guardrails

Проблема: пользователь может прислать запрос, который не по теме ("как варить борщ"). Мы не хотим тратить на него ресурсы.

Guardrail — защитный слой, который до поиска проверяет: "это вообще про нашу тему?".

У нас реализован через **LLM-классификатор**: даём llama-ке промпт "отнеси запрос к fashion или нет, верни JSON" — если нет, возвращаем ошибку.

Fail-open — важный паттерн: если guardrail (LLM) упал, **пропускаем запрос** (а не блокируем). Лучше изредка пропустить мусор, чем регулярно ломать UX для нормальных юзеров.

1.11 Query reformulation

Проблема: CLIP учили в основном на английском. На русском он тоже работает, но с потерей качества.

Решение: прежде чем искать, просим LLM переписать запрос в 3 английских варианта.

- "чёрное платье" → ["black dress", "black gown", "black evening dress"]

Для каждого варианта — свой поиск, результаты объединяем. Плюсы:

- Картиночная часть CLIP работает на **английском**, как привыкла.
- 3 варианта = больше recall (охватываем разные формулировки).

1.12 Метрики качества поиска

- **Recall@K** — "из всех релевантных документов, сколько % попало в топ-K". Важно для покрытия.
- **Precision@K** — "из топ-K, сколько % релевантных". Важно для UX (первый экран результатов).
- **MRR (Mean Reciprocal Rank)** — средняя величина $1/\text{позиция}$ первого правильного ответа. Хорошо, когда важен именно первый хит.
- **nDCG** — сложная метрика, учитывает и релевантность, и позицию.

Мы меряем **Precision@5** — "из первых 5 результатов, сколько релевантных".

1.13 FastAPI

Фреймворк для Python для создания HTTP API. Почему он:

- **Быстрый** (основан на Starlette + uvicorn, асинхронный).
- **Pydantic под капотом** — автоматическая валидация входных данных по схеме.
- **Авто-документация** (Swagger UI на /docs).
- Популярен в ML-команиях.

Endpoint — URL, обрабатывающий запросы: `POST /search`, `POST /search/image`.

1.14 Observability (наблюдаемость)

Проблема: сервис крутится в проде, что-то тормозит. Как понять, где и почему?

Три "столпа":

1. **Logs** (логи) — текстовые записи о событиях. `logger.info("user searched")`.
2. **Metrics** (метрики) — числа во времени: RPS, latency, error rate. Агрегированные.
3. **Traces** (трассировки) — цепочки вызовов в распределённых системах. (У нас нет, не нужны для монолита.)

1.15 Prometheus

Time-series БД, специально для метрик. Как работает:

1. Приложение выставляет эндпоинт `/metrics` — там текстом написаны текущие значения всех метрик.
2. Prometheus раз в N секунд ходит туда (**pull model**, в отличие от push).
3. Сохраняет у себя → можно строить графики.

Типы метрик в Prometheus:

- **Counter** (счётчик) — только растёт: `http_requests_total`. Скорость = `rate(counter[5m])`.
- **Gauge** (датчик) — может расти и падать: `memory_used_mb`.
- **Histogram** (гистограмма) — распределение значений по бакетам. Для латентности — единственно правильный выбор.

Почему histogram для latency:

- Mean (среднее) врёт на хвостах. 99 запросов по 10мс + 1 запрос на 10сек → mean = 110мс, выглядит норм, но 1% пользователей страдает.
- Histogram даёт **перцентили**: p50 (медиана), p95, p99. p99 = 10сек сразу видно.

1.16 Grafana

Веб-инструмент для **визуализации метрик**. Читает из Prometheus, строит дашборды: графики RPS, латентности, ошибок.

Grafana сама ничего не хранит — это "морда" поверх БД.

1.17 Structlog

Python-библиотека для **структурированных логов**. Обычный `logger.info("search done in 150ms, user=42")` — это текст, его тяжело парсить.

structlog пишет JSON: `{"event": "search_done", "duration_ms": 150, "user_id": 42}`. Такие логи легко парсятся агрегаторами (Loki, ELK).

У нас через structlog пробрасывается **request_id** — уникальный ID, который присваивается на входе запроса и виден во всех логах одного запроса. Если юзер жалуется — по request_id собираешь всю цепочку.

1.18 Unit-тесты и моки (stubs)

Unit-тест — проверка одной маленькой функции изолированно. "Поддай вход X, ожидай выход Y".

Проблема: наш код импортирует `sentence_transformers` (2GB torch), `fastembed`, `qdrant_client`. В тесте мы не хотим реально грузить CLIP.

Заглушка (stub/mock) — фальшивый объект с тем же интерфейсом. Подсовываем его вместо настоящей библиотеки через `sys.modules`. Код думает, что работает с реальной моделью, а под капотом — простая заглушка, возвращающая фиксированные значения.

Итог: 23 теста проходят за 0.6 секунды без скачивания моделей.

1.19 CI (Continuous Integration)

Идея: каждый коммит/PR автоматически проверяется — собирается ли код, проходят ли тесты.

GitHub Actions — бесплатный CI от GitHub. В файле `.github/workflows/ci.yml` описываешь шаги:

1. Развернуть Ubuntu-виртуалку.
2. Поставить Python.
3. Поставить зависимости.
4. Запустить линтер (ruff).
5. Запустить pytest.

Если что-то падает — CI красный, видно в PR. Не даём смержить сломанный код.

1.20 Docker и Docker Compose

Docker-контейнер — изолированный мини-Linux с приложением и всеми его зависимостями. "Работает у меня на ноуте" → "работает везде".

Dockerfile — рецепт сборки контейнера (какую базу брать, что поставить, что запустить).

Docker Compose — описываешь в YAML несколько контейнеров, которые работают вместе. У нас 4:

- qdrant — векторная БД,
- app — наш FastAPI,
- prometheus — сбор метрик,
- grafana — дашборды.

`docker-compose up -d` — поднимает всё одной командой.

Часть 2. Архитектура — как всё связано

Этап А: Индексация (один раз, офлайн)

Скрипт `build_index.py`:

1. Скачивает датасет из HuggingFace (fashion-товары: картинка + текстовое описание + метаданные color/gender/category).
2. Для каждого товара:
 - Прогоняет картинку через **CLIP** → получает dense-вектор 512 чисел.
 - Прогоняет текстовое описание через **BM42** → получает sparse-вектор.
 - Сохраняет в **Qdrant** как одну точку с двумя именованными векторами (dense, sparse) + payload (цвет, категория, пол, путь к картинке).
3. Результат: коллекция `fashion` с ~5000 точек.

Важно: индексация — дорогая операция (требует GPU/долго на CPU). Делается один раз, потом поиск использует готовый индекс.

Этап В: Онлайн — запрос пользователя

Юзер отправляет `POST /search` с телом `{"query": "чёрное платье"}`.

Шаг 1. Middleware (`observability.py` + `api.py`)

- Сгенерировали `request_id` (UUID).
- Записали время старта.
- Пробросили `request_id` в контекст всех логов.

Шаг 2. Guardrail (`llm.py`)

- Запрос идёт в LLM с промптом "это fashion?".
- LLM возвращает JSON `{"is_fashion": true/false}`.
- Если `false` → возвращаем 400 ошибку.
- Кешируем в `@lru_cache(512)` — тот же запрос второй раз не грузит LLM.
- Если LLM упала — **fail-open**, пропускаем.

Шаг 3. Query reformulation (`llm.py`)

- LLM берёт русский запрос → возвращает JSON со списком из 3 английских парафразов.
- "чёрное платье" → `["black dress", "dark dress", "black evening dress"]`.

- Если LLM не отвечает — fallback на исходный запрос (fail-open).

Шаг 4. Hybrid search — для каждого из 3 запросов (search.py)

- Текст → dense-вектор через CLIP (multilingual text encoder).
- Текст → sparse-вектор через BM42.
- **Один** запрос в Qdrant с обоими векторами + **RRF fusion** на сервере.
- Получаем top-20 кандидатов.

Шаг 5. Union запросов

3 списка по 20 → объединяем (убирая дубли), усредняем скоры.

Шаг 6. Rerank (search.py)

- Берём кандидатов (до 20 штук).
- Пары (query_english, product_description) → **cross-encoder** (mmarco-mMiniLMv2, multilingual).
- Получаем точный скор для каждой пары.
- Сортируем заново по новому скору.

Шаг 7. Ответ

Топ-К (обычно 5-10) возвращается клиенту:

```
{
  "results": [
    {"id": "123", "score": 0.87, "payload": {...}, "image_url": "/image"},
  ],
  "request_id": "abc-def-..."
}
```

Шаг 8. Observability

На каждом шаге `with timed("stage_name")`: пишет в Prometheus histogram. Prometheus раз в 10с забирает `/metrics`. Grafana показывает графики.

Этап С: Поиск по картинке — `/search/image`

То же самое, но:

- Нет guardrail / reformulation (картинка — самодостаточна).
- CLIP-image-encoder превращает её в dense-вектор.
- Только dense-поиск (sparse для картинок не имеет смысла).
- Rerank опционален (cross-encoder текстовый, не подходит для картинок напрямую).

Часть 3. Почему именно такие решения

Почему Qdrant, а не FAISS/Weaviate/pgvector?

- **FAISS** — библиотека, не сервис. Нет фильтров по payload, надо писать обёртку.
- **pgvector** (Postgres) — проще на старте, но нет native hybrid search, sparse не поддерживает.
- **Weaviate** — ок, но heavier, GraphQL вместо REST.
- **Qdrant**: нативный hybrid (dense + sparse в одной точке), RRF на сервере, Rust (быстро), простой REST.

Почему hybrid, а не только dense?

Dense пропускает:

- Точные бренды/артикулы ("Nike Air Max 90").
- Редкие слова, которых мало в обучении.

Sparse пропускает:

- Синонимы ("кофта" vs "свитер").
- Смысл при разной формулировке.

Гибрид покрывает обе дыры.

Почему cross-encoder после bi-encoder?

- Cross-encoder в 100+ раз медленнее на запрос, но **заметно точнее**.
- На миллионе документов его напрямую не применить — будет минуты.
- Двухступенчато: bi-encoder миллион → 20, cross-encoder 20 → 5. Лучшее из двух миров.

Почему reformulate RU→EN?

- CLIP сильнее на английском (обучался преимущественно на en-парах).
- 3 парафразы = больше recall.
- Стоимость минимальна: Cerebras отвечает за ~200мс.

Почему Cerebras?

- **2000+ tok/s**. OpenAI — 50-100. Groq — ~500. Cerebras — быстрее всех.
- LLM перестаёт быть bottleneck в пайплайне.
- Бесплатно для dev-использования.

Почему Prometheus + Grafana (а не, скажем, Datadog)?

- Open-source, self-hosted, бесплатно.
- Стандарт индустрии, указание в резюме понятно любому SRE.
- Легко поднимается в docker-compose.

Почему structlog, а не обычный logging?

- JSON-логи парсятся любым агрегатором.
- Context propagation (request_id) — из коробки.
- Код тот же: `log.info("event", key=value)`.

Почему моки/заглушки в тестах, а не реальные модели?

- Реальный CLIP = 2GB + скачать в CI. Минуты на каждый прогон.
- Заглушка = секунды.
- Unit-тест проверяет **нашу логику**, а не "работает ли CLIP" (это забота Meta).

Почему Docker Compose?

- Один `docker-compose up` — вся система: БД + API + Prometheus + Grafana.
 - Ревьюер/работодатель может запустить у себя за 2 минуты.
 - Воспроизводимость: "на моей машине работает" → "работает у всех".
-

Часть 4. Что рассказать на собеседовании

Elevator pitch (30 секунд)

Мультимодальный поисковик для каталога одежды. Пользователь вводит текст на русском или английском (либо загружает картинку) — получает релевантные товары. Под капотом — **гибридный retrieval** (dense + sparse) в Qdrant с **cross-encoder reranking**, LLM для **query reformulation** русского в английский (CLIP сильнее на английском) и **guardrails**. Обёрнуто в FastAPI, есть **Prometheus/Grafana** для observability, **unit-тесты** с заглушками тяжёлых моделей, **CI на GitHub Actions**.

Если спросят "а в чём вклад, это же всё готовые библиотеки?"

- Сборка пайплайна — это не библиотека. Знаю, **почему** каждый шаг нужен, **какие альтернативы** были.
- Решил конкретные проблемы: низкие скоры реранкера на русском → поменял на multilingual + reformulate.
- Сделал production-обёртку: observability, guardrails, fail-open, тесты, CI, docker-compose.

Типичные вопросы и ответы

Q: Зачем вообще reranker, если dense + sparse уже хорош? A: Bi-encoder сжимает документ в один вектор — детали теряются. Cross-encoder смотрит пару запрос-документ целиком, ловит тонкие взаимодействия слов. На тестах давал +N% к precision@5.

Q: Почему RRF, а не взвешенная сумма скоров? A: Шкалы dense (0-1 косинус) и BM25 (0-50) несравнимы. Взвешенная сумма требует тюнить веса под датасет. RRF опирается только на ранг → устойчив, без тюнинга.

Q: Как измеряешь качество? A: Синтетические triple-ы (query, positive, negative) — LLM генерирует запросы к случайным товарам, меряем Precision@5. Честная оговорка: это не реальные логи, но даёт архитектурный сигнал.

Q: Что будет на миллионе товаров? A: Bi-encoder с HNSW-индексом — $O(\log N)$, ок. Cross-encoder на 20 кандидатов — не зависит от размера коллекции. Узкое место — GPU для CLIP при индексации, либо batching.

Q: Как мониторишь в проде? A: Prometheus собирает histogram на каждую стадию (retrieval, rerank, llm, total). В Grafana смотрю p95/p99, rate ошибок LLM, rate guardrail-

отказов. Через structlog все логи в JSON с request_id для трейсинга.

Q: Почему тесты не грузят реальные модели? А: Unit-тесты проверяют нашу логику, не stability CLIP. Заглушки через `sys.modules` — тесты за 0.6с. Интеграционные (с реальным Qdrant) — отдельно, через testcontainers (пока не добавлено, но план есть).

Честные ограничения (говорить первым, не ждать вопроса)

- Eval на **синтетике**, не на реальных пользовательских логах → числа показывают архитектурную валидность, не продакшн-качество.
- Нет **auth/rate-limit** — MVP.
- Reranker на CPU — ок для 5к документов, для продакшна с миллионами нужен GPU.
- Нет **fine-tuning** CLIP на fashion-домен — brand-specific термины могут теряться.
- Нет **интеграционных тестов** с живым Qdrant (только unit).

Что это показывает работодателю

1. Понимаю RAG end-to-end, не просто "позвал OpenAI API".
2. Разбираюсь в retrieval: dense vs sparse, ранги vs скоры, bi vs cross.
3. Думаю про продакшн: observability, guardrails, fail-open, тесты, CI.
4. Умею объяснять tradeoffs: почему этот инструмент, а не другой.
5. Hands-on с Docker, FastAPI, GitHub Actions — стандартный стек.

Приложение: мини-гlossарий одной строкой

- **RAG** — поиск + LLM-ответ по найденному.
- **Embedding** — вектор чисел, кодирующий смысл.
- **Dense** — плотный вектор из нейросети, ловит смысл.
- **Sparse** — разреженный, ловит точные слова.
- **BM25 / BM42** — формулы sparse-релевантности.
- **CLIP** — модель текст + картинка в одном пространстве.
- **Bi-encoder** — быстро считает вектор запроса и документа отдельно.
- **Cross-encoder** — медленно, но точно: смотрит пару целиком.
- **RRF** — объединение рейтингов через обратный ранг.
- **Qdrant** — векторная БД с hybrid search.
- **LLM** — большая языковая модель.
- **Guardrail** — защитный фильтр до поиска.
- **Reformulate** — переписать запрос для лучшего поиска.
- **Prometheus** — сбор метрик.
- **Grafana** — визуализация метрик.
- **Histogram** — тип метрики для распределения значений (даёт перцентили).
- **p95 / p99** — "95% / 99% запросов быстрее чем...".
- **Structlog** — JSON-логи.
- **Mock / stub** — фальшивый объект для тестов.
- **CI** — автопроверка кода на каждый коммит.
- **Docker Compose** — запуск нескольких контейнеров вместе.